

Machine Learning



Dr. Ahmed Hesham Mostafa
Faculty of Computers and AI
Helwan University
Lecture 1

Course Staff: Instructors (Short Bio)

- **Dr. Ahmed Hesham Mostafa**
- Bachelor in Computer Science *from* Helwan University
- Masters in Computer Science (Malware Analysis & Artificial Intelligence) *from* Helwan University
- PhD. in Computer Science (Computer Vision & Artificial Intelligence) *from* Helwan University
- E-mail : ahmedheshamostafa@gmail.com

Course Objectives

By the end of this course, students will be able to:

Understand the Fundamentals

1. Explain the core concepts of machine learning, including supervised, unsupervised, semi-supervised, reinforcement, and self-supervised learning.
2. Differentiate between traditional programming and machine learning approaches.

Apply Key Algorithms

1. Implement and analyze major ML algorithms (e.g., linear regression, logistic regression, decision trees, SVMs, clustering, and neural networks).
2. Evaluate algorithm performance using appropriate metrics.

Work with Data

1. Preprocess, clean, and transform raw data into usable formats for machine learning models.
2. Apply feature engineering and dimensionality reduction techniques.

Model Training & Evaluation

1. Train, validate, and test machine learning models using real-world datasets.
2. Apply methods to prevent overfitting and improve generalization (e.g., cross-validation, regularization).

Course Objectives

Tools & Frameworks

1. Use popular ML libraries and frameworks (e.g., scikit-learn, TensorFlow).
2. Apply best practices for reproducibility and experimentation.

Critical Thinking & Problem Solving

1. Select appropriate ML approaches for different types of problems (classification, regression, clustering, etc.).
2. Interpret results and critically assess model limitations.

Project Work

1. Design and implement a complete ML pipeline (data collection, preprocessing, modeling, evaluation, and deployment).
2. Present results effectively through reports and presentations.

Course Contents

- **Introduction to machine Learning**
- **Linear Regression and Introduction to optimization and Gradient Descent**
- **Logistic Regression and Entropy**
- **Decision Trees and random Forrest**
- **Naïve Bayes Classifier**
- **Support Vector Machine**
- **Clustering and K-means**
- **Model Performance Metrics**
- **Artificial Neural Networks**
- **Optimization Algorithms**
- **Loss Function Gradient**

Course Materials

- **References :**
 - Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
 - Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- **Interactive Book**
- **Lecture Slides**
- **For Practice**
 - [GitHub - ageron/handson-ml3: A series of Jupyter notebooks that walk you through the fundamentals of Machine Learning and Deep Learning in Python using Scikit-Learn, Keras and TensorFlow 2.](#)
 - [GitHub - maykulkarni/Machine-Learning-Notebooks: Machine Learning notebooks for refreshing concepts.](#)
 - [Machine Learning | Google for Developers](#)

Section

- Section will cover
- NumPy → For numerical operations (arrays, matrices).
- Pandas → For data manipulation and cleaning (tables, CSVs).
- Matplotlib & Seaborn → For data visualization (graphs, plots).
- **Scikit-learn (sklearn)**
 - Covers all classical algorithms: regression, classification, clustering, model evaluation.

Grades

- 20% Midterm
- 30% Project
 - Classification Task
 - Regression Task
 - Clustering Task

What is Machine Learning?

- Machine learning is an application of AI that provides systems the ability to learn on their own and improve from experiences without being programmed externally. by feeding it large amounts of data.
- Machine learning allows computer systems to continuously adjust and enhance themselves as they accrue more “experiences.” Thus, the performance of these systems can be improved by providing larger and more various datasets to be processed.
- If your computer had machine learning, it might be able to play difficult parts of a game or solve a complicated mathematical equation for you.

What is Machine Learning?

- Machine learning (ML) works by teaching computers to **learn patterns from data** rather than being programmed with step-by-step instructions.
- **The Core Idea**
 - Traditional programming: You write rules → computer applies them to data → produces output.
 - Machine learning: You feed the computer data and examples → it figures out the rules (patterns/statistical relationships) by itself → then applies them to new data.

How Machine Learning Works

Key Components

1. Data – The raw material (numbers, text, images, etc.).

Example: To teach a computer to recognize cats, you give it many labeled pictures of cats and non-cats.

2. Features – The characteristics the algorithm looks at.

Example: In images: edges, shapes, textures. In finance: income, spending, credit score.

3. Model – A mathematical structure that learns relationships in the data.

Example: Linear regression (straight line), decision trees, or deep neural networks.

4. Training – The process of adjusting the model's parameters so it fits the data well.

Uses optimization techniques (like **gradient descent**) to minimize “errors.”

5. Prediction/Inference – Once trained, the model can make predictions on new, unseen data.

How Machine Learning Works

- Each step is important and cannot be skipped to achieve high accuracy. Code Implementation follows the below-mentioned steps:
 - Data collection
 - Data Preprocessing
 - Model Training
 - Model Evaluation
 - Model Deployment

How Machine Learning Works

- **Data Collection:** Data collection is an important part as discussed above.
- The quality of data determines the accuracy of the predictions. We can collect datasets from APIs, websites, social media, etc.
- We can also use in-built datasets that are provided in the programming languages for learning purposes.
- Ethical use of data should be kept in mind.

How Machine Learning Works

- **Data Preprocessing:** Before feeding this data into modeling we pre-process this data to remove duplicate and missing values, deal with outliers, and standardize the formats is done in this step.
- This enhances the quality of the dataset and improves accuracy by dealing with the possible error sources before modeling.

How Machine Learning Works

- **Model Training:** After we get the dataset we use an algorithm based on our problem to model our dataset.
- We usually divide our dataset into two parts: training and testing sets. Various models are used, for example, linear regression, logistic regression, decision trees, etc.
- **Hyperparameter** training is also done to improve accuracy. Techniques like **grid search** and **random search** are used for tuning the parameter.

How Machine Learning Works

- **Model Evaluation:** This is a Critical step in determining whether our model is working accurately or not.
- Metrics such as accuracy, precision, recall, F1-score, and AUC guide the assessment of model performance.
- Cross-validation techniques like k-fold and leave-one-out help us in determining the efficiency of the model.
- These values are the determinants of the accuracy.

How Machine Learning Works

- **Model Deployment:** Deployment of these steps into real-world problems is done in this step.
- It is a process of integrating the trained model into real-world issues to solve them.
- This is the practical use of the model building and training.

How Machine Learning Works

- Other than these steps we also visualize our predictions as well as accuracy to get a better understanding of our model.
- For example, we can plot feature importance plots to understand which particular feature plays the most important role in altering the predictions.
- This helps in feature selection and engineering.

Machine Learning Classification

- The most common and useful classification breaks machine learning into three (or five) primary categories.
 - **Supervised Learning**
 - **Unsupervised Learning**
 - **Reinforcement Learning**
 - **Semi-Supervised Learning**
 - **Self-Supervised Learning**

Supervised Learning

- Supervised Learning is a type of machine learning where an algorithm learns to map input data (X) to known output labels (y). It's called "supervised" because the process of an algorithm learning from a labeled dataset is analogous to a teacher supervising a student.
- The model is trained on a dataset that includes both the input examples and the correct, desired output. The goal is for the model to learn the underlying patterns or rules so it can accurately predict the outputs for new, unseen data.
- Think of it like a **teacher supervising a student**: the student (algorithm) gets examples with the right answers, learns from them, and then tries to answer new questions.

The Core Idea: Learning with a Teacher

- Imagine you're teaching a child to identify different fruits. You show them an apple and say, "This is an apple." You show them a banana and say, "This is a banana." After many examples, you show them a new fruit they haven't seen, and they can (hopefully) identify it correctly.
- This is the core of supervised learning. The algorithm is the "child," the data is the "fruits," and the labels ("apple," "banana") are the "teacher's" guidance. The goal is to learn the mapping between the inputs (fruits) and the outputs (names) so it can predict the label for new, unseen data.

Supervised learning Multi-stage process

Supervised learning works in a clear, multi-stage process:

1. Data Collection and Preparation

This is the most critical step. You need a **dataset**. This dataset must be:

- **Labeled:** Each example (data point) has both:
 - **Features (Input variables):** The measurable characteristics or attributes.
 - **Label (Target output):** The answer we want to predict.
- **Example:** A dataset for predicting house prices.
 - **Features:** Number of bedrooms, square footage, zip code, age of the house.
 - **Label:** The actual sale price of the house.

Supervised learning Multi-stage process

2. Choosing a Model

A model is a mathematical algorithm that will learn from the data. The choice depends on the problem.

- **Simple Problems:** Linear Regression, Logistic Regression.
- **Complex Problems:** Decision Trees, Support Vector Machines, Neural Networks.

Supervised learning Multi-stage process

3. Training: The Learning Phase This is where the magic happens. The algorithm is fed the training data (features and their corresponding labels).

The Process:

- The model makes a prediction based on the input features.
- It compares its prediction to the actual known label.
- It calculates the error (the difference between the prediction and the truth). This is done using a **loss function** (e.g., Mean Squared Error for regression).
- The model then adjusts its internal parameters (e.g., weights in an equation) to reduce this error.

This process repeats again and again for all the data points in the training set, often over multiple iterations (called **epochs**). The goal is to find the best possible parameters that minimize the overall prediction error.

Analogy: It's like tuning a radio. You start with static (high error), and you slowly turn the dial (adjust parameters) until the signal comes in clear (error is minimized).

Supervised learning Multi-stage process

- 4. Evaluation: Testing the Model** After training, we must test the model's performance on **new, unseen data** (the test set). This is critical to ensure it learned general patterns and didn't just **memorize** the training data (a problem called **overfitting**).

We use metrics like:

- **Accuracy:** For classification (e.g., what percentage of emails did it correctly classify as spam/not spam?).
- **Mean Squared Error (MSE):** For regression (e.g., how far, on average, are the predicted house prices from the real prices?).

5. Prediction (Inference)

Once the model is trained and evaluated to a satisfactory level, it is deployed to make predictions on **brand-new, real-world data** where the label is unknown.

- **Example:** You input the features of a new house (4 bedrooms, 2000 sq ft, etc.) into the trained model, and it outputs a predicted price

Types of Supervised Learning

- Types of Supervised Learning
 - Classification
 - Regression

Types of Supervised Learning

Classification – Predicting a *category* (discrete output).

- Examples:
 - Email → Spam / Not Spam
 - Medical diagnosis → Sick / Healthy
 - Handwriting recognition → Digits 0–9
- Algorithms: Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, Neural Networks.

Types of Supervised Learning

Regression – Predicting a *continuous value*.

- Examples:
 - Predicting house prices
 - Estimating stock prices
 - Forecasting temperatures
- Algorithms: Linear Regression, Polynomial Regression, Support Vector Regression (SVR), Neural Networks.

Key Concepts and Challenges

- **Overfitting**: The model learns the training data too well, including its noise and random variation. It becomes like a student who memorizes the textbook answers but fails the exam because the questions are phrased differently. It performs well on training data but poorly on new data.
- **Underfitting**: The model is too simple to capture the underlying pattern in the data. It's like a student who didn't study enough. It performs poorly on both training and new data.
- **Generalization**: This is the ultimate goal. It refers to the model's ability to perform accurately on new, unseen data after being trained on the training data.

Unsupervised Learning

- The key idea is **finding hidden patterns in unlabeled data**.
- The algorithm is given data without any predefined labels and must find structure on its own.
- It's about exploring the data and inferring the natural patterns within it.
- The algorithm tries to discover hidden patterns, structures, or relationships within the data.
- Analogy: Giving a student a set of different objects (e.g., fruits, vegetables, books) without any labels and asking them to sort them into groups based on their similarities.

Unsupervised Learning

Common Algorithms:

- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- Association Rules (e.g., Apriori algorithm)

Types of Unsupervised Learning

- Clustering
- Dimensionality Reduction
- Association Rule Learning

Types of Unsupervised Learning

Clustering

- Group data points into clusters based on similarity.
- Examples:
 - Customer segmentation (grouping shoppers by buying behavior)
 - Image grouping (organizing photos by content)
 - Document/topic clustering
- Algorithms: **K-Means, Hierarchical Clustering**

Types of Unsupervised Learning

Dimensionality Reduction

- Reduce the number of features while keeping important information.
- Examples:
 - Visualizing high-dimensional data in 2D/3D
 - Compressing data for storage or faster computation
 - Noise reduction in images
- Algorithms: **PCA (Principal Component Analysis)**

Types of Unsupervised Learning

Association Rule Learning

- Discover rules that describe relationships in data.
- Examples:
 - Market basket analysis → "People who buy bread often buy butter"
 - Recommender systems (Amazon, Netflix suggestions)
- Algorithms: **Apriori, FP-Growth**

Reinforcement Learning

The key idea is **learning through interaction and feedback**. An "agent" learns to make decisions by performing actions in an "environment" to achieve a goal. It learns from the consequences of its actions (rewards or penalties), rather than from being taught explicitly.

- **Analogy:** Teaching a dog a new trick. The dog (agent) tries different actions (sitting, rolling over). You (the environment) give it a treat (positive reward) for the correct action and nothing or a gentle correction (negative reward) for incorrect actions. Over time, the dog learns which action maximizes its treats.
- **Common Algorithms/Methods:** Q-Learning

Reinforcement Learning

Applications:

- Teaching a robot to walk.
- Developing strategies for board games like Chess and Go (e.g., AlphaGo).
- Self-driving cars making real-time decisions.
- Managing inventory in warehouses.

Semi-Supervised Learning (A Hybrid Approach)

- This type sits between supervised and unsupervised learning. It uses a small amount of labeled data and a large amount of unlabeled data to train models. This is extremely useful because, in the real world, obtaining labeled data is often expensive and time-consuming, while unlabeled data is cheap and abundant.
- Analogy: A student is given a history textbook with a few chapters highlighted and summarized by the teacher (labeled data) and many more chapters without any notes (unlabeled data). The student uses the summarized chapters to understand how to interpret the unsummarized ones.

Self-Supervised Learning

- **Self-Supervised Learning (SSL)** is one of the most exciting areas in modern machine learning, especially behind systems like **GPT**, **BERT**, **CLIP** and others.
- It's a type of machine learning that sits between supervised and unsupervised learning.
- In SSL, the system generates its own labels from raw data instead of relying on human-labeled datasets.
- The model is trained with a pretext task (an artificial prediction task) where part of the input is hidden, and the model must predict it from the rest.

Self-Supervised Learning

Example:

- Data : “The cat sat on the mat.”
- Give a model a sentence: “The cat ____ on the mat.”
- Hide a word and ask it to predict the missing word (“**sat**”).
- Then compare the prediction with original if “incorrect” repeat and so on
- No human labels required — the data itself creates the supervision.

Self-Supervised Learning

Why “Self-Supervised”?

- **Supervised:** Needs external labels (e.g., cat/dog).
- **Unsupervised:** No labels at all.
- **Self-Supervised:** The model **creates supervision from the input data itself.**

Self-Supervised Learning Training

- Training a self-supervised learning (SSL) model is a two-phase process: first **pre-training** on a pretext task to learn general representations, followed by **fine-tuning** on a downstream task to apply that knowledge.

Self-Supervised Learning Training

Phase 1: Pre-training on the Pretext Task

This is the core "self-supervised" phase where the model learns from unlabeled data.

Step 1: Gather a Large Unlabeled Dataset

The first step is to collect a massive dataset of raw, unlabeled data. This is SSL's biggest advantage—this data is cheap and abundant.

- **For Images:** Millions of images from the internet (e.g., Instagram, Flickr).
- **For Text:** Billions of web pages, books, articles (e.g., Wikipedia, Common Crawl).
- **For Video:** Millions of YouTube videos.

Self-Supervised Learning Training

- **Phase 1: Pre-training on the Pretext Task**

Step 2: Define and Apply the Pretext Task

This is the creative heart of SSL. You design a task where the label is automatically generated from the data itself. The model will be trained to solve this task, but the real goal is for it to learn useful features *as a byproduct*.

Common Pretext Tasks include:

- **Masked Autoencoding:** Randomly obscure parts of the input and train the model to reconstruct the missing parts.
 - **NLP (Masked Language Modeling):** Mask 15% of the words in a sentence; task is to predict the masked words. (*Used by BERT*)
 - **Computer Vision:** Mask random patches of an image; task is to predict the missing pixels. (*Used by MAE*)
- **Rotation Prediction (CV):** Rotate an image by 0°, 90°, 180°, or 270°; task is to predict the rotation applied.

Self-Supervised Learning Training

- **Phase 1: Pre-training on the Pretext Task**

Step 3: Choose a Model Architecture

Select a neural network architecture suitable for your data and task.

- **For Images: A Vision Transformer (ViT) or a Convolutional Neural Network (CNN) like ResNet.**
- **For Text: A Transformer architecture (e.g., the encoder from the Transformer model, used in BERT).**

Self-Supervised Learning Training

- **Phase 1: Pre-training on the Pretext Task**

Step 4: Train the Model on the Pretext Task

This is the intensive computational step. The model is trained on the pretext task using standard deep learning techniques:

- **Input:** The transformed data from the pretext task (e.g., a masked sentence, an augmented image).
- **Forward Pass:** The model makes a prediction (e.g., the missing word, the rotation angle).
- **Loss Calculation:** A loss function calculates how wrong the prediction is compared to the "pseudo-label" (e.g., the actual masked word, the actual rotation angle).
- **Backpropagation:** The error is propagated backward through the network, and the model's parameters (weights) are adjusted to minimize the loss.
- The model repeats this process over millions of examples until the representations it learns become robust and general.

Self-Supervised Learning Training

- **Outcome of Phase 1:** You now have a **pre-trained model** that has learned high-quality, general-purpose feature representations of your data domain (e.g., images, language). It is not yet useful for any specific business task, but it is an extremely powerful feature extractor.

Self-Supervised Learning Training

Phase 2: Fine-Tuning on a Downstream Task

Now, we take the general knowledge from the pre-trained model and apply it to a task we actually care about, for which we usually have a limited amount of *labeled* data.

Step 5: Prepare a Labeled Dataset for the Downstream Task

Collect a (much smaller) dataset for your specific problem. This dataset *must be labeled*.

- **Example Task:** Classify images as "cat" or "dog".
- **Example Dataset:** 10,000 images that are manually labeled as either "cat" or "dog".

Self-Supervised Learning Training

- **Phase 2: Fine-Tuning on a Downstream Task**

Step 6: Modify the Model Architecture

Take your pre-trained model and add a new **task-specific head** on top of it. The "head" is typically one or more new layers that map the general features to the specific output you need.

- For a classification task, you would remove the final layer used for the pretext task (e.g., the word-prediction layer) and replace it with a new **classification layer** (e.g., a linear layer that outputs probabilities for "cat" and "dog").

Self-Supervised Learning Training

- **Phase 2: Fine-Tuning on a Downstream Task**

Step 7: Fine-Tune the Model

Train the *entire* modified model on your new, labeled downstream task. The key here is **transfer learning**:

- **Initialization:** The model starts with all the weights learned from the large-scale pre-training, not random weights. This gives it a massive head start.
- **Process:** You train the model on your downstream dataset. The early layers (which detect basic features like edges or word stems) may only need slight adjustments, while the newer layers and later stages of the model learn task-specific patterns.
- **Learning Rate:** A common practice is to use a very low **learning rate** to gently adjust the pre-trained weights without destroying the valuable representations they contain.

Self-Supervised Learning Training

- **Phase 2: Fine-Tuning on a Downstream Task**

Step 8: Evaluate and Deploy

- **Evaluate:** Test the fine-tuned model on a held-out **test set** to measure its performance on the downstream task (e.g., classification accuracy).
- **Deploy:** If performance is satisfactory, deploy the model for real-world use.

References

- [What is Machine Learning? Types and uses | Google Cloud](#)
- [An Introduction To Machine Learning](#)
- [How does Machine Learning Works? – GeeksforGeeks](#)